

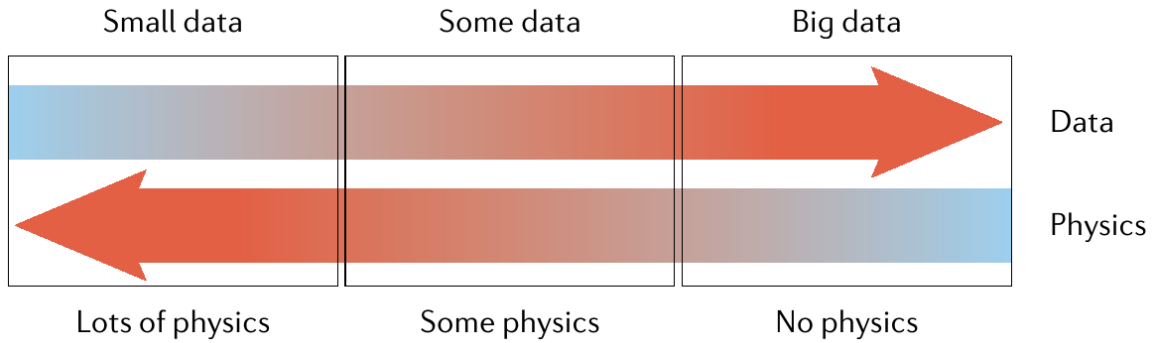
ML FOR SCIENCE · LECTURE 14

Physics-Informed ML

Aparna Chandramowlishwaran · EECS 195/221 · Fall 2026

Training a model without solver data

- Lectures 12-13: FNO, DeepONet and GNNs learned from **solver data**. The Burgers operator was trained on 1000 simulated trajectories
- Each training sample costs **one solver run**. Measurements from experiments are often **sparse and noisy**
- We usually know the **governing equation**. Physics-informed learning adds it to the **loss**
- Three regimes:
  - *Lots of physics, small data*: solve or invert one PDE
  - *Some physics, some data*: combine sparse measurements with the equation
  - *No physics, big data*: learn from data alone



Today: networks trained on the equation

Karniadakis, Kevrekidis, Lu, Perdikaris, Wang & Yang, Physics-informed machine learning, Nature Reviews Physics 2021

Three ways to add physics to a model

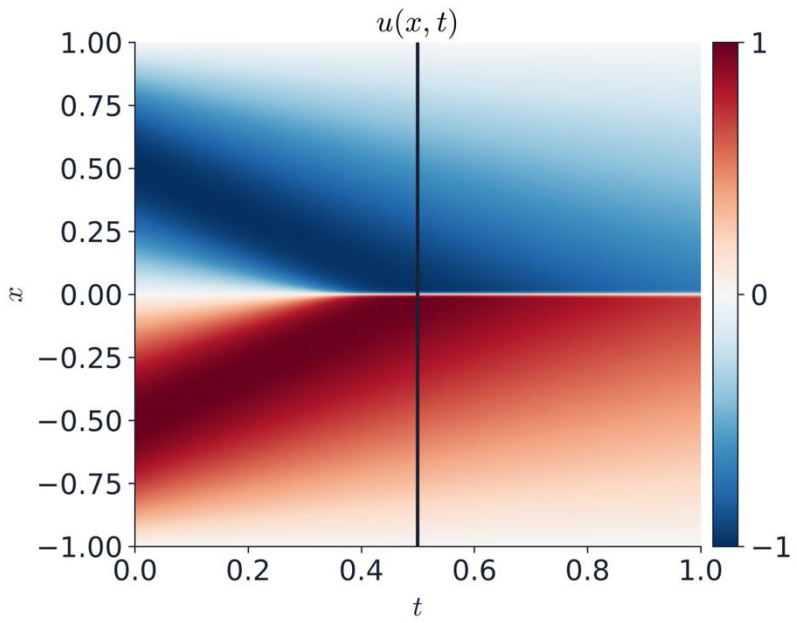
	How the physics enters	Example in this course
Observational bias	Training data that obey the physics: simulations, transformed samples	Rotated and reflected samples (Lecture 5)
Inductive bias	The architecture satisfies the physics exactly	Equivariant CNNs (Lecture 5), conservation by construction (Lecture 15)
Learning bias	A penalty in the loss for violating the physics	PDE residual loss (today)

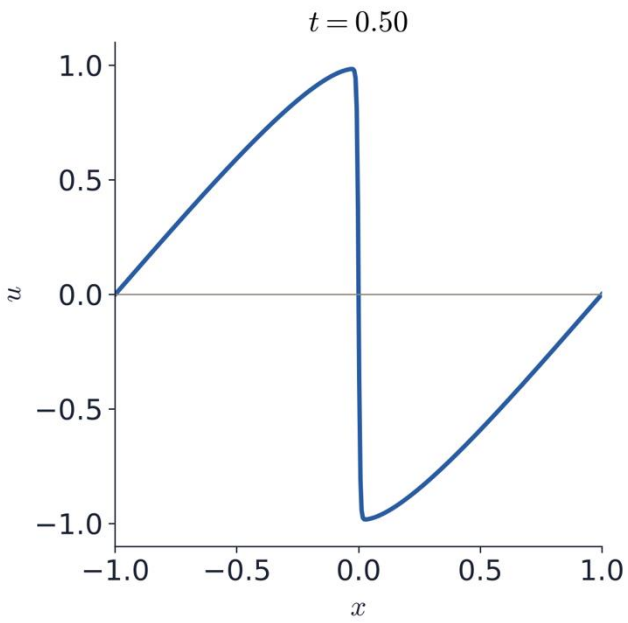
- Observational and inductive biases leave the loss unchanged. A learning bias changes the **loss**
- A penalty is a **soft constraint**: the trained model satisfies the physics only **approximately**
- Lecture 15 builds **hard constraints** that hold exactly

Karniadakis, Kevrekidis, Lu, Perdikaris, Wang & Yang, Physics-informed machine learning, Nature Reviews Physics 2021

Burgers' equation as the running example

$$\frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} = \nu\,\frac{\partial^2 u}{\partial x^2}, \qquad u(x,0) = -\sin(\pi x), \qquad u(\pm 1, t) = 0$$





- $u(x, t)$: velocity; $x \in [-1, 1]$, $t \in [0, 1]$; viscosity $\nu = 0.01/\pi$
- The term $u\,\partial u/\partial x$ steepens the sine into a **shock** at $x = 0$. Shock width is proportional to ν.
- Lecture 12 trained on Burgers with $\nu = 0.01$ and random initial conditions. Today, **one fixed** initial condition
- Reference: the exact solution from the **Cole-Hopf** transform

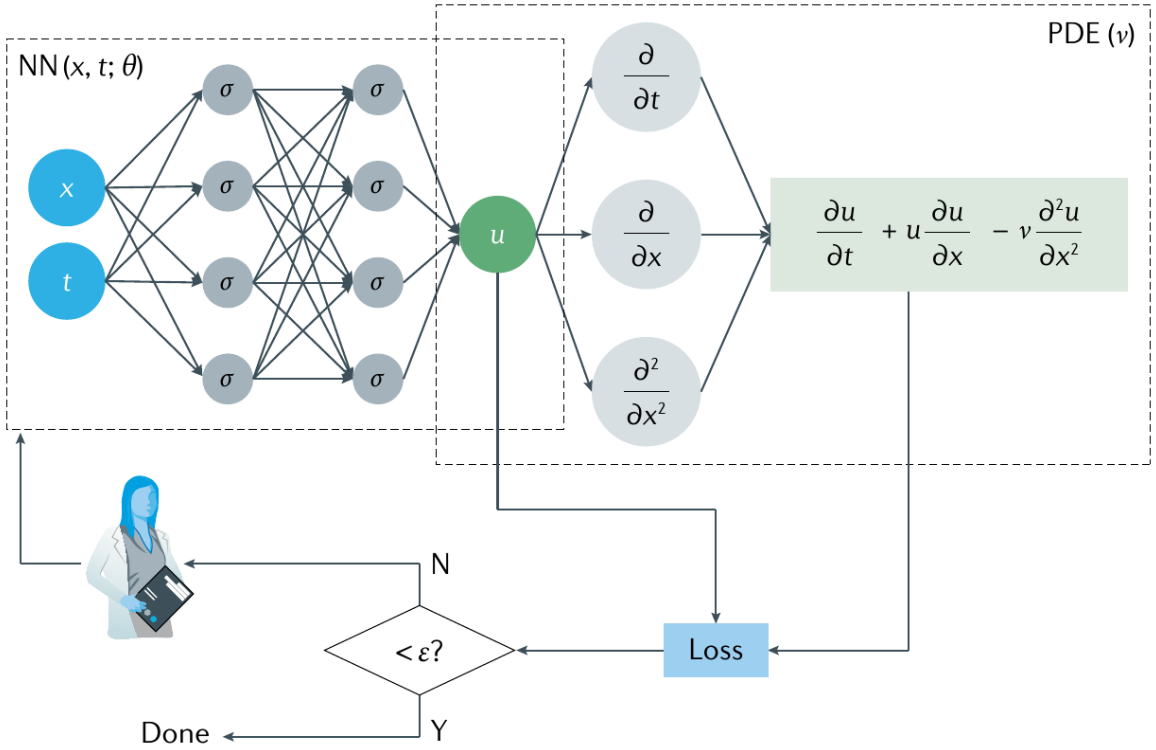
Raissi, Perdikaris & Karniadakis, Physics-informed neural networks, Journal of Computational Physics 2019. Basdevant et al., Computers & Fluids 1986

Network represents the solution

- A **neural field** $u_\theta(x, t)$ takes coordinates and returns the field value
- Automatic differentiation gives **exact** derivatives of u_θ with respect to x and t
- Insert them into the PDE to get the **residual**:

$$r_\theta(x,t) = \frac{\partial u_\theta}{\partial t} + u_\theta \frac{\partial u_\theta}{\partial x} - \nu \frac{\partial^2 u_\theta}{\partial x^2}$$

- $r_\theta = 0$ everywhere when u_θ solves the PDE; $|r_\theta|$ measures how far it is from a solution
- Figure: network (left), derivatives by autodiff (middle), residual and loss (right)



Karniadakis, Kevrekidis, Lu, Perdikaris, Wang & Yang, Physics-informed machine learning, Nature Reviews Physics 2021. Raissi et al., JCP 2019

Physics-informed loss

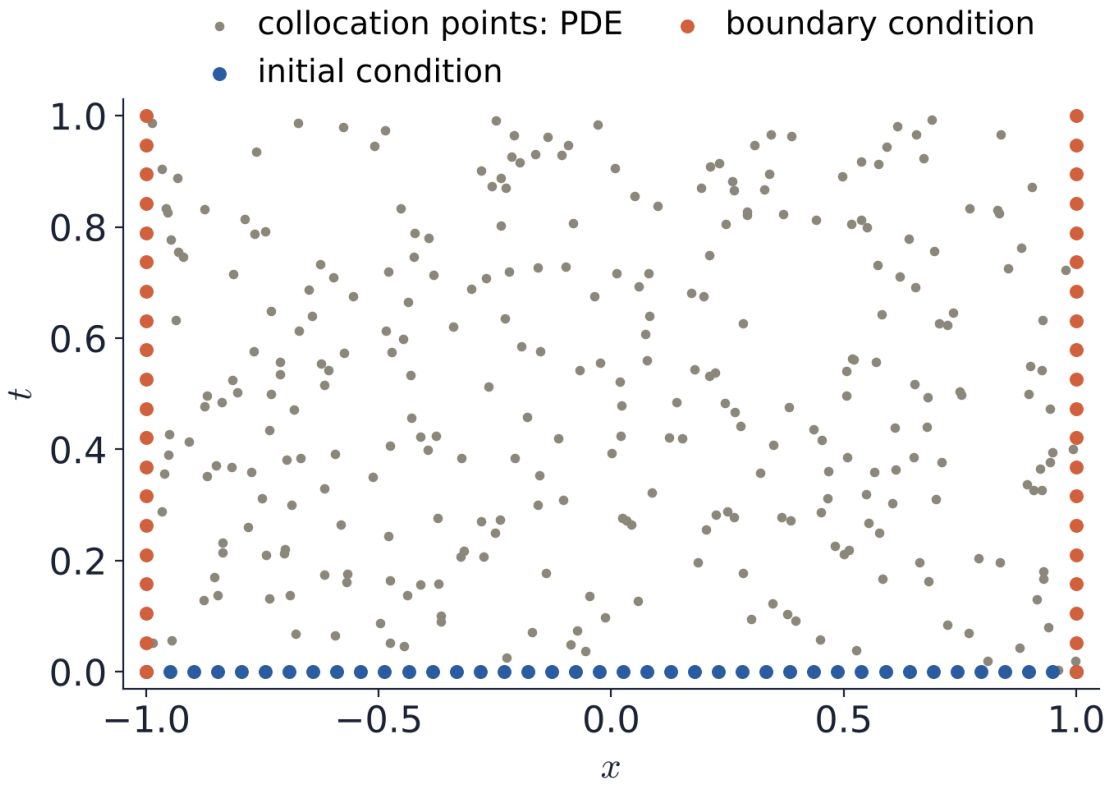
$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N_r}\sum_{i=1}^{N_r} r_\theta(x_i, t_i)^2}_{\text{PDE}} + \underbrace{\frac{1}{N_0}\sum_{j=1}^{N_0} \left(u_\theta(x_j, 0) - u_0(x_j)\right)^2}_{\text{initial condition}} + \underbrace{\frac{1}{N_b}\sum_{k=1}^{N_b} u_\theta(\pm 1, t_k)^2}_{\text{boundary condition}}$$

- **Collocation points** (x_i, t_i) lie inside the domain. PDE residual is evaluated at the collocation points. **No solution values** are needed
- **Initial and boundary points** enforce u_0 and $u(\pm 1, t) = 0$
- Without these terms, many functions have zero residual ($u = 0$ solves Burgers). The conditions select **the** solution (well-posedness)
- Train θ with Adam, then **L-BFGS**, a quasi-Newton method that builds a curvature estimate from recent gradients
- Lagaris et al. (1998) proposed the idea; Raissi et al. (2019) combined autodiff with deep networks: the **physics-informed neural network (PINN)**

Zero loss: network satisfies the PDE, initial and boundary conditions at the sampled points

Lagaris, Likas & Fotiadis, IEEE Transactions on Neural Networks 1998. Raissi et al., JCP 2019

Collocation points replace the mesh



- Points are sampled at random in space and time (uniform or Latin hypercube). Raissi et al. sampled 10000 for Burgers
- No mesh or grid: **irregular geometries** and **higher dimensions** only require sampling points in the new domain
- Time is an input like x: PINN solves for **all times at once**, without time stepping
- Resample points during training. Adding points where $|r_\theta|$ is large (**residual-based adaptive refinement**) helps near shocks
- PDE is checked only at the points. With too few points, the network can satisfy it at the collocation points and **violate it between them**

Lu, Meng, Mao & Karniadakis, DeepXDE: a deep learning library for solving differential equations, SIAM Review 2021

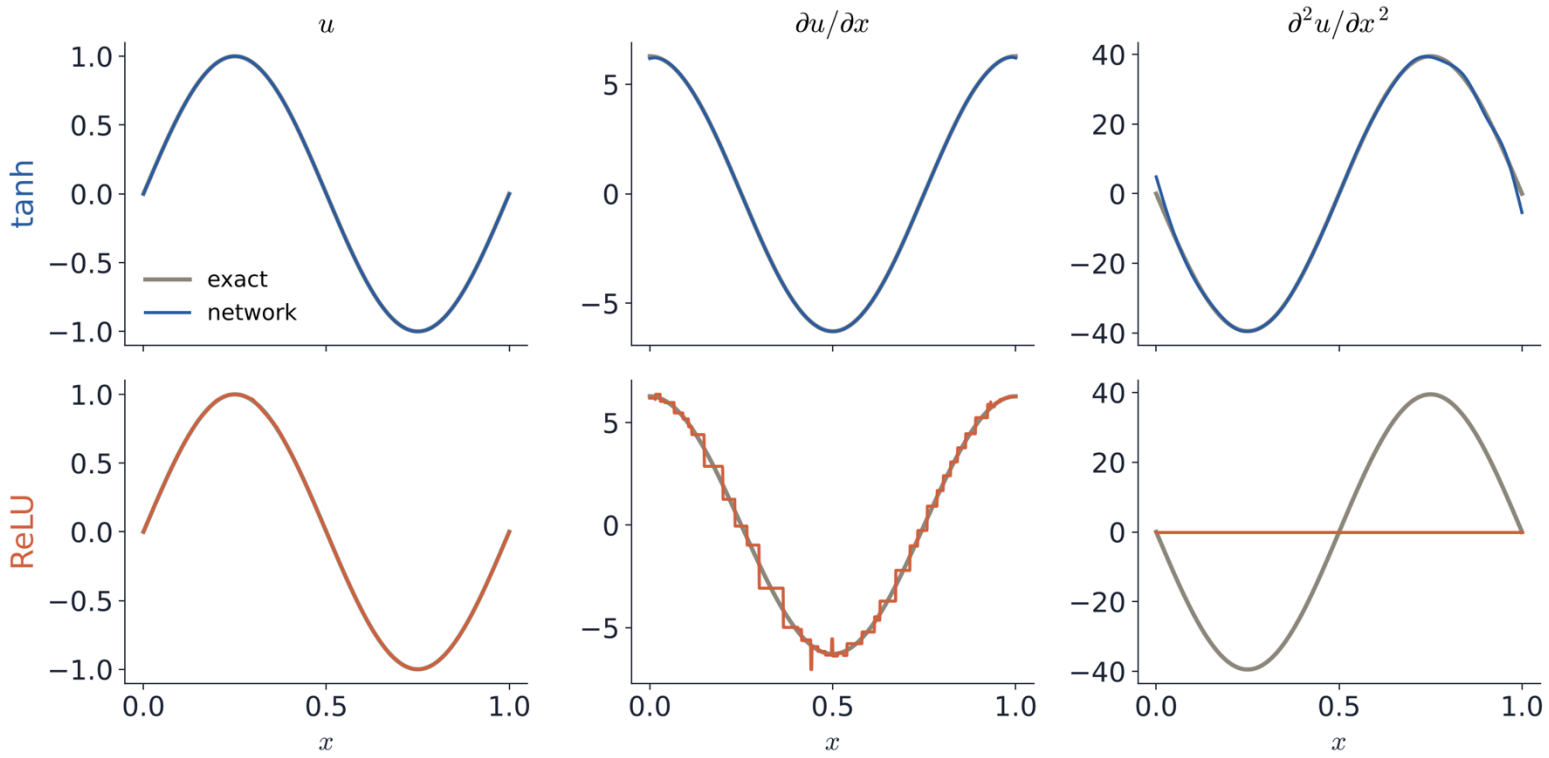
Derivatives with respect to the inputs

```
def D(f, v):                    # df/dv at every point
    return torch.autograd.grad(f, v, torch.ones_like(f),
                               create_graph=True)[0]

x.requires_grad_(True); t.requires_grad_(True)
u = net(x, t)
u_t, u_x = D(u, t), D(u, x)
u_xx = D(u_x, x)
r = u_t + u * u_x - nu * u_xx     # residual
loss = (r**2).mean() + loss_ic + loss_bc
loss.backward()                   # gradient with respect to θ
```

- Backpropagation differentiates with respect to the **weights**. PINN also needs derivatives with respect to the **inputs**
- *create_graph=True* adds each derivative to the computation graph, so *loss.backward()* can differentiate through it
- Each derivative is another backward pass: one training step costs **several times** a step that fits data only

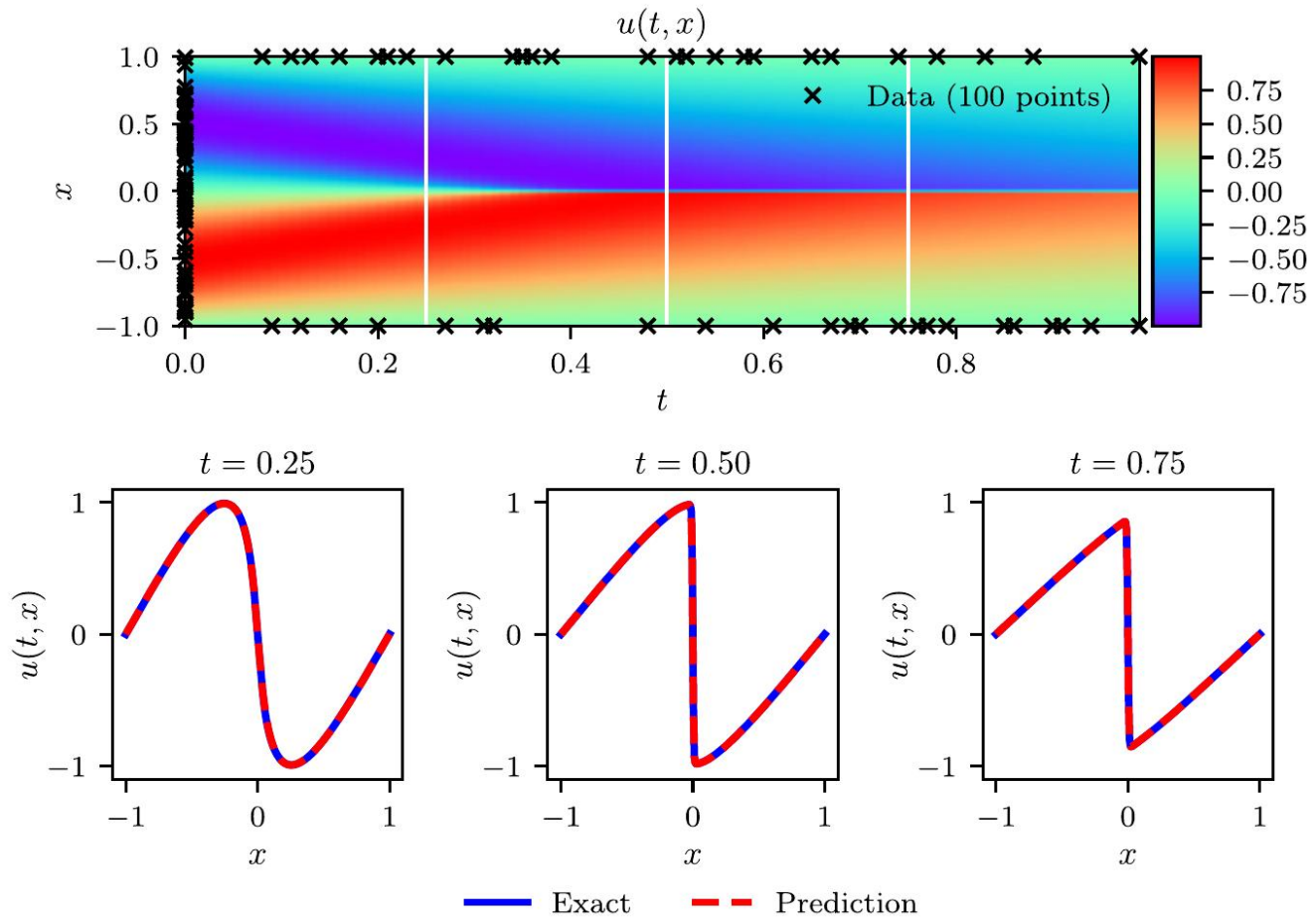
Activation needs nonzero second derivatives



- Two networks with one hidden layer of 60 neurons, fit to $\sin(2\pi x)$
- **ReLU**: u is piecewise linear, $\partial u/\partial x$ is a staircase, and $\partial^2 u/\partial x^2 = 0$ between the kinks
- The diffusion term $\nu\, \partial^2 u/\partial x^2$ is then **zero at all collocation points**
- Use **tanh**, **sine** or smooth variants (GELU, swish)

The network must be differentiable as many times as the PDE requires

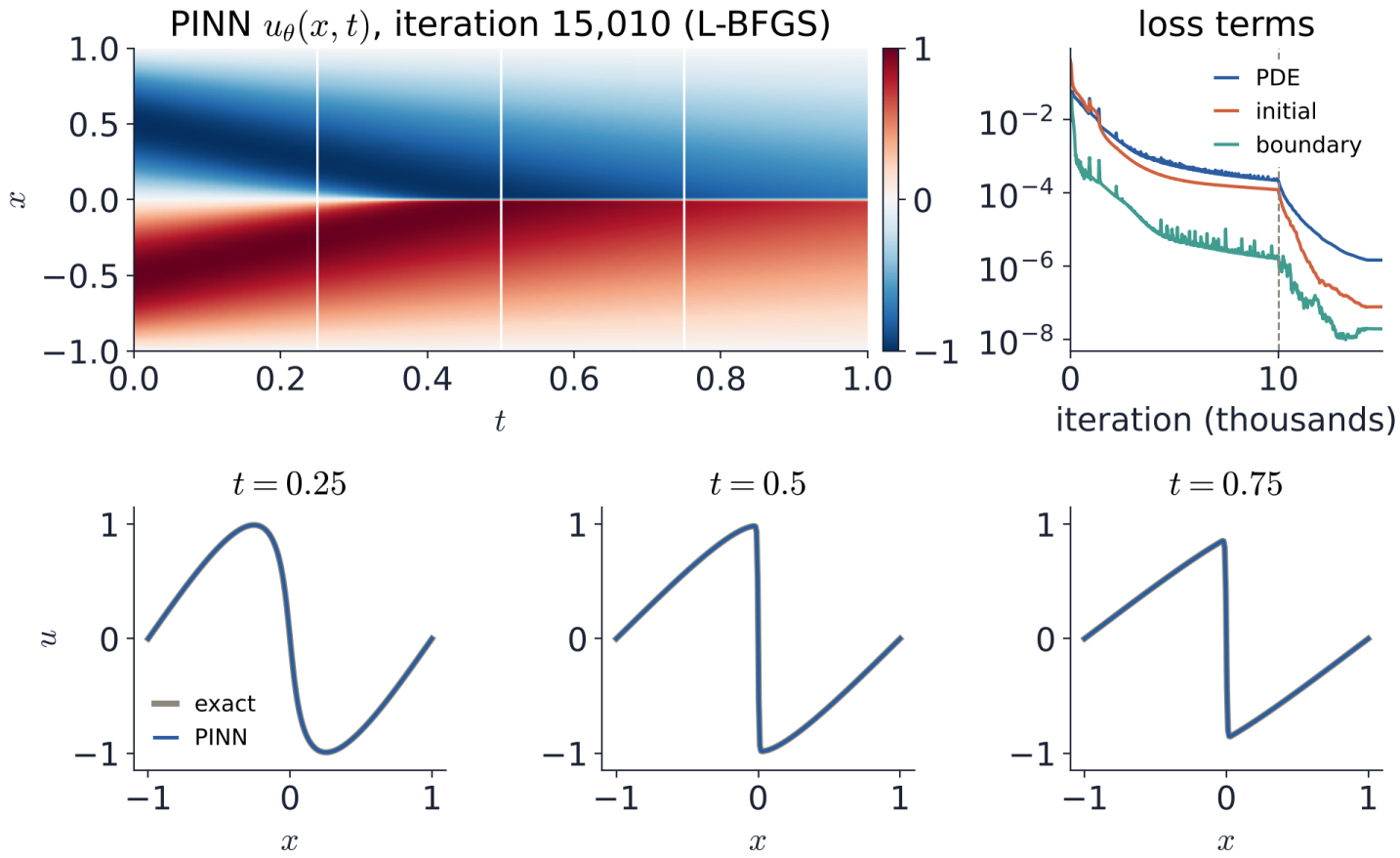
PINN solution of Burgers' equation



- Network: 9 layers of 20 tanh neurons. **100** initial and boundary points (× in the top panel), **10000** collocation points, L-BFGS
- Top: the predicted $u(t, x)$. Bottom: prediction (red) and exact solution (blue) at three times
- Prediction matches the exact solution, **including the shock**, with relative error of order 10^{-3}
- No solution values inside the domain: the interior comes from the **PDE alone**

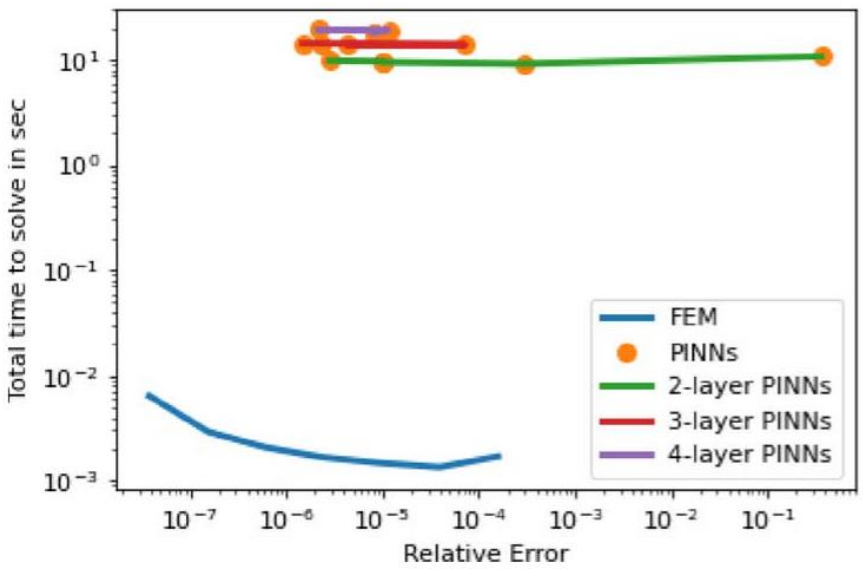
Raissi, Perdikaris & Karniadakis, Physics-informed neural networks, Journal of Computational Physics 2019

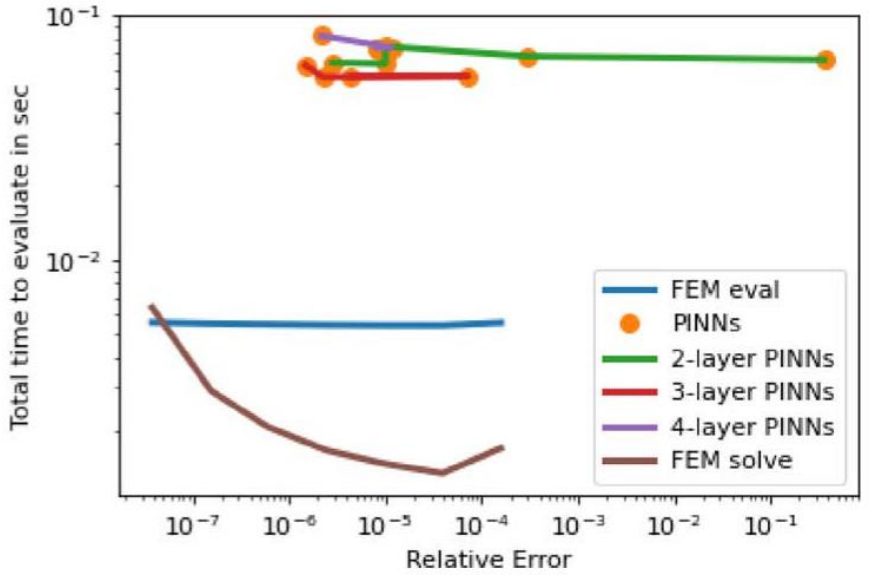
Solution emerges during training



- The network first fits a **smooth** version of the solution
- Smooth regions converge early; the error then concentrates at the **shock**, the **last feature resolved**
- Adam lowers the loss quickly, then slows; **L-BFGS** lowers it by orders of magnitude and sharpens the shock

Cost compared with the finite element method





- Poisson, Allen-Cahn and Schrödinger in 1D-3D: FEM was **faster and more accurate** in their tests (left)
- A trained PINN evaluates quickly (right), but **training** dominates the cost
- A PINN solves **one instance**: a new initial condition, coefficient or geometry requires re-training
- Lecture 10: compare at equal accuracy against an efficient solver

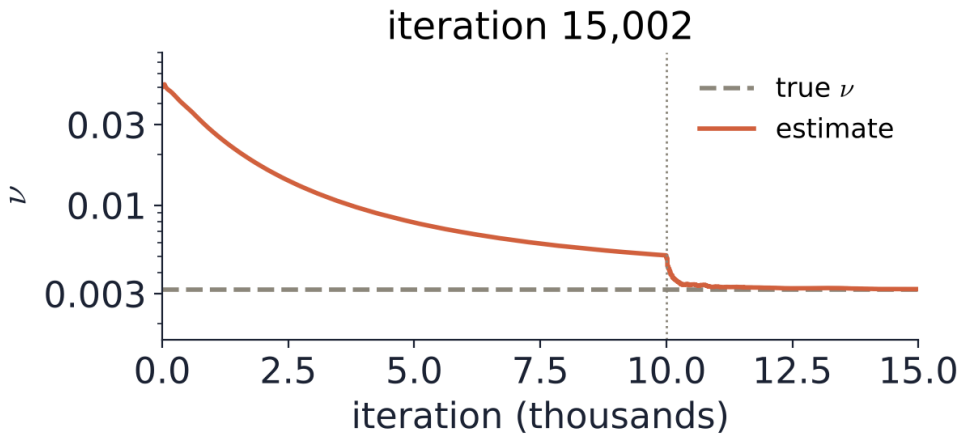
For a forward problem with known coefficients, a numerical solver is usually the better tool

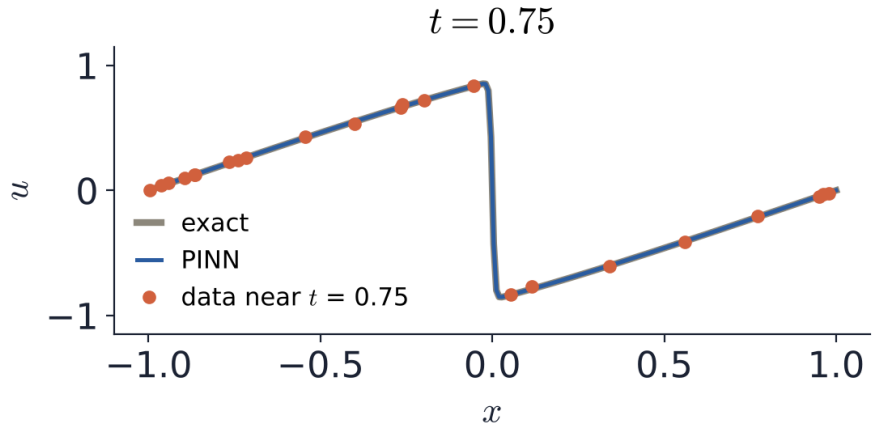
Grossmann, Komorowska, Latz & Schönlieb, Can physics-informed neural networks beat the finite element method?, IMA Journal of Applied Mathematics 2024

Inverse problems: learning an unknown coefficient

$$\mathcal{L}(\theta, \nu) = \frac{1}{N_d} \sum_{i=1}^{N_d} \left(u_\theta(x_i, t_i) - u_i \right)^2 + \frac{\lambda}{N_r} \sum_{j=1}^{N_r} r_{\theta,\nu}(x_j, t_j)^2$$

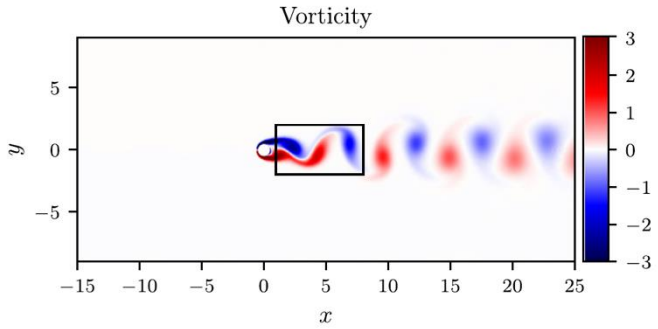
- The viscosity ν is **unknown**. We have $N_d = 500$ **noisy measurements** u_i at scattered (x_i, t_i), and no initial or boundary data
- Make ν a **trainable parameter** along with θ: one loss and one optimizer update both
- Data term fits the measurements. Residual makes u_θ obey Burgers with the current ν
- A classical approach wraps a solver in an optimization loop over ν. PINN fits the field and ν in **one training run**

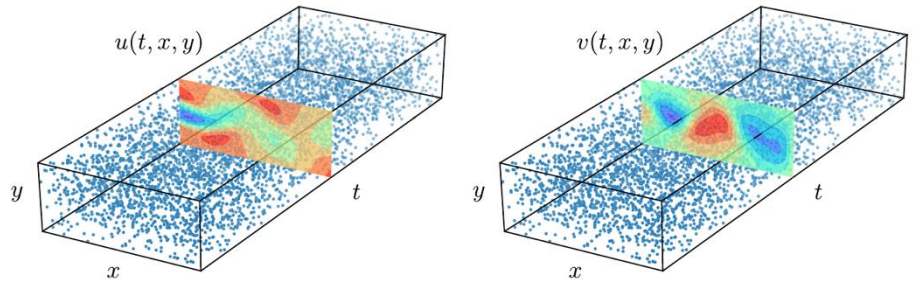


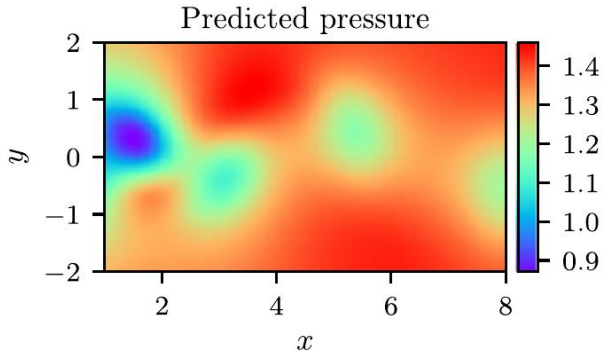


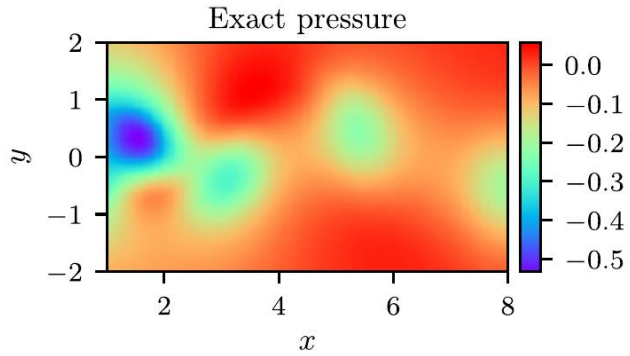
Raissi, Perdikaris & Karniadakis, Physics-informed neural networks, Journal of Computational Physics 2019

Identifying Navier-Stokes from velocity data





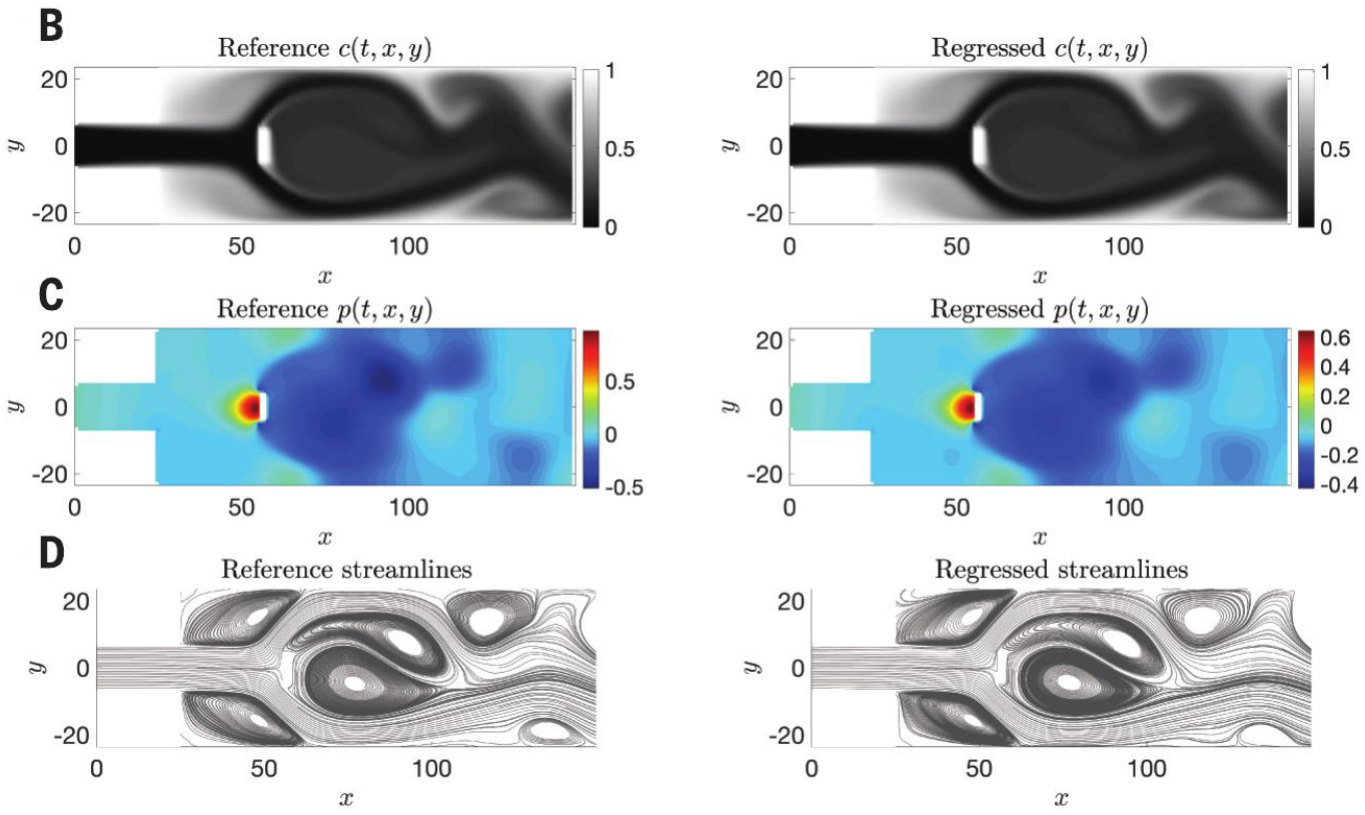




Correct PDE	$u_t + (uu_x + vu_y) = -p_x + 0.01(u_{xx} + u_{yy})$ $v_t + (uv_x + vv_y) = -p_y + 0.01(v_{xx} + v_{yy})$
Identified PDE (clean data)	$u_t + 0.999(uu_x + vu_y) = -p_x + 0.01047(u_{xx} + u_{yy})$ $v_t + 0.999(uv_x + vv_y) = -p_y + 0.01047(v_{xx} + v_{yy})$
Identified PDE (1% noise)	$u_t + 0.998(uu_x + vu_y) = -p_x + 0.01057(u_{xx} + u_{yy})$ $v_t + 0.998(uv_x + vv_y) = -p_y + 0.01057(v_{xx} + v_{yy})$

- Flow past a cylinder at *Re* = 100. Data: velocity (*u*, *v*) at **5000** random points in the wake, about 1% of the simulation data (left)
- Unknown coefficients λ_1, λ_2 in $u_t + \lambda_1(u\,u_x + v\,u_y) = -p_x + \lambda_2(u_{xx} + u_{yy})$, and the same for *v*
- Recovered $\lambda_1 \approx 1$ and $\lambda_2 \approx 0.01$ (table), and the **pressure field** with **no pressure data**, up to a constant (top right)

Velocity and pressure from a dye concentration

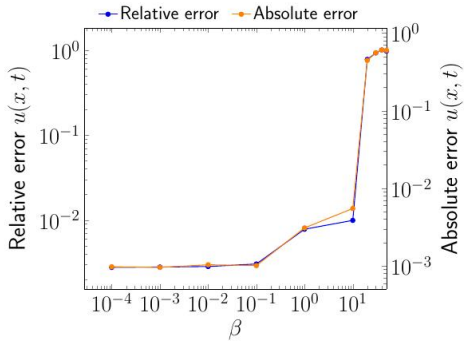


- Dye or smoke is easy to photograph. Velocity and pressure are hard to measure
- The dye concentration c is carried by the flow: $c_t + \mathbf{u}\cdot\nabla c = Pe^{-1}\nabla^2 c$
- Network: $(t, x, y) \rightarrow (c, u, v, p)$. Loss: data on **$c$ only**, plus residuals of the dye, momentum and continuity equations
- Velocity, pressure and streamlines are recovered (left: reference vs. inferred)
- Also applied to blood flow in a 3D brain aneurysm

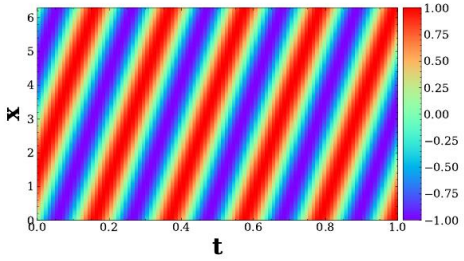
Raissi, Yazdani & Karniadakis, Hidden fluid mechanics, Science 2020

Failure on convection with a large speed

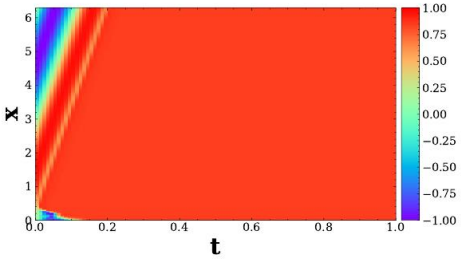
$$\frac{\partial u}{\partial t} + \beta \frac{\partial u}{\partial x} = 0, \qquad u(x,0) = \sin x, \qquad x \in [0, 2\pi] \text{ periodic}$$



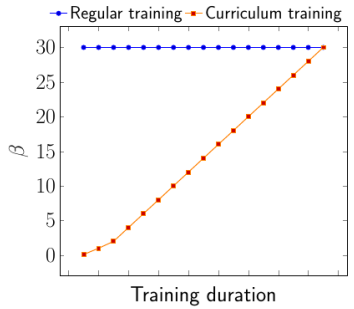
(a) *Error for different* β



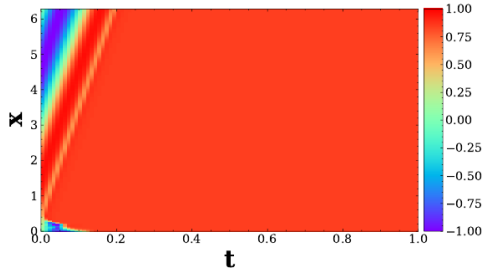
(b) *Exact solution for* $\beta = 30$



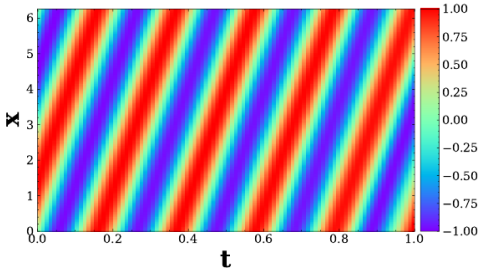
(c) *PINN solution for* $\beta = 30$



(a) *Curriculum regularization schematic*



(b) *Regular training PINN solution for* $\beta = 30$

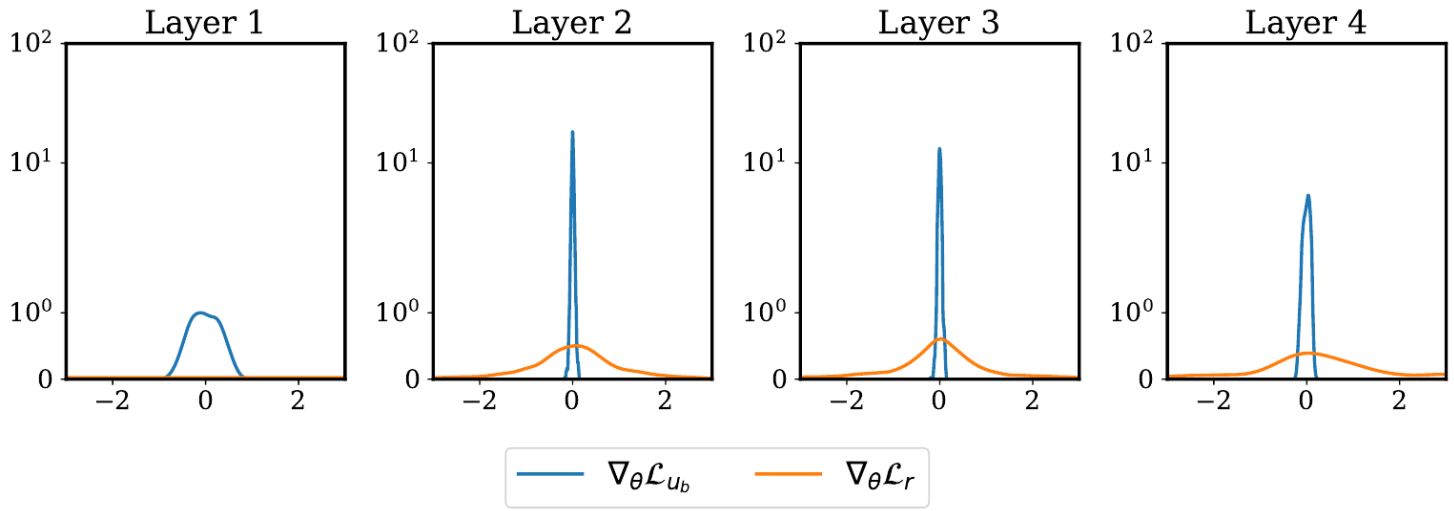


(c) *Curriculum training PINN solution for* $\beta = 30$

- Exact solution: $u = \sin(x - \beta t)$, a smooth traveling wave a network can represent
- Small β: small error. $\beta = 30$: the PINN returns a **nearly constant** field (top)
- Loss landscape gets rougher as β grows: an **optimization** failure
- **Curriculum**: train at small β, then increase β step by step from the previous weights (bottom)

Krishnapriyan, Gholami, Zhe, Kirby & Mahoney, Characterizing possible failure modes in physics-informed neural networks, NeurIPS 2021

Loss terms with unbalanced gradients



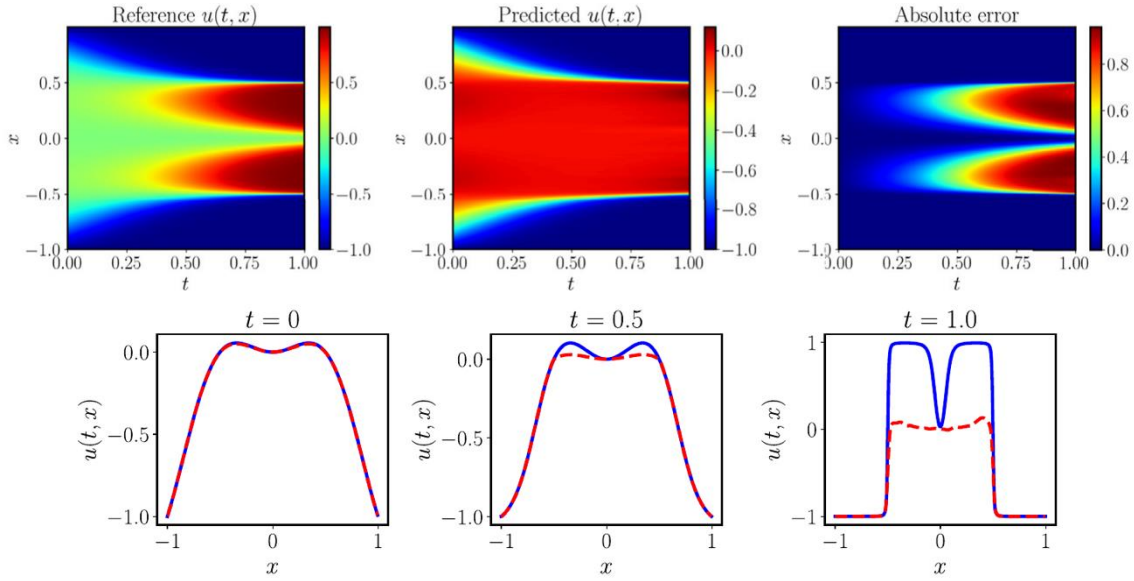
- Helmholtz equation: gradients of the residual loss $\mathcal{L}_r$ (orange) are much **larger** than those of the boundary loss $\mathcal{L}_b$ (blue, peaked at 0)
- Training follows $\mathcal{L}_r$ and **ignores the boundary conditions**

$$\mathcal{L}(\theta) = \mathcal{L}_r(\theta) + \lambda_b \mathcal{L}_b(\theta), \qquad \hat{\lambda}_b = \frac{\max_\theta |\nabla_\theta \mathcal{L}_r|}{\overline{|\nabla_\theta (\lambda_b \mathcal{L}_b)|}}, \qquad \lambda_b \leftarrow (1-\alpha)\,\lambda_b + \alpha\,\hat{\lambda}_b$$

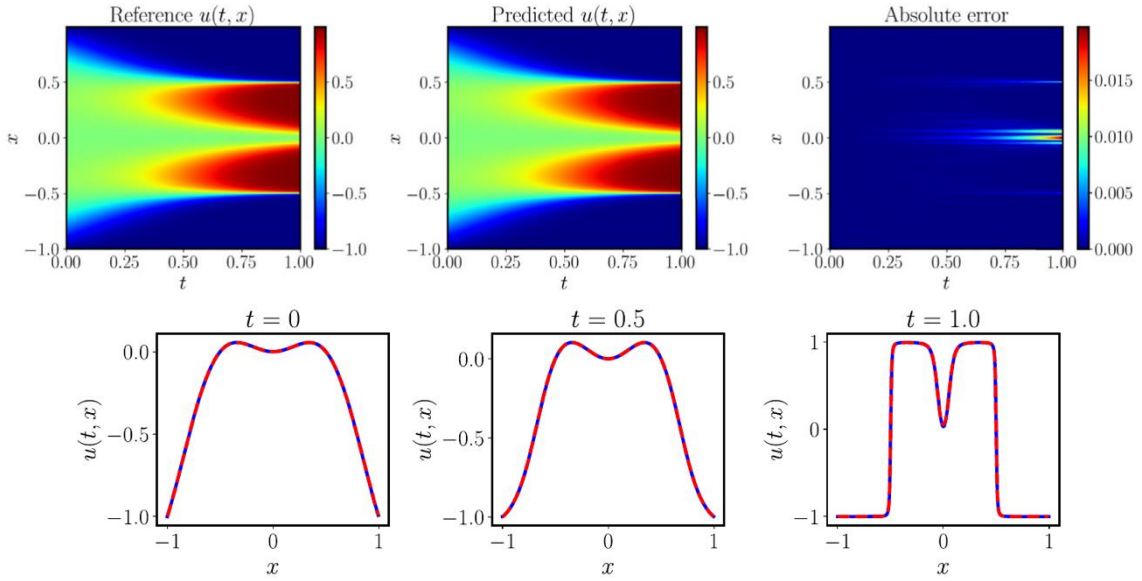
- **Adaptive weights**: rescale λ_b during training so the gradient magnitudes balance; $\alpha \approx 0.1$ smooths the update
- Related: weights from the neural tangent kernel of each term (Wang, Yu & Perdikaris 2022)

Wang, Teng & Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM J. Sci. Comput. 2021

Training that respects causality



Standard training



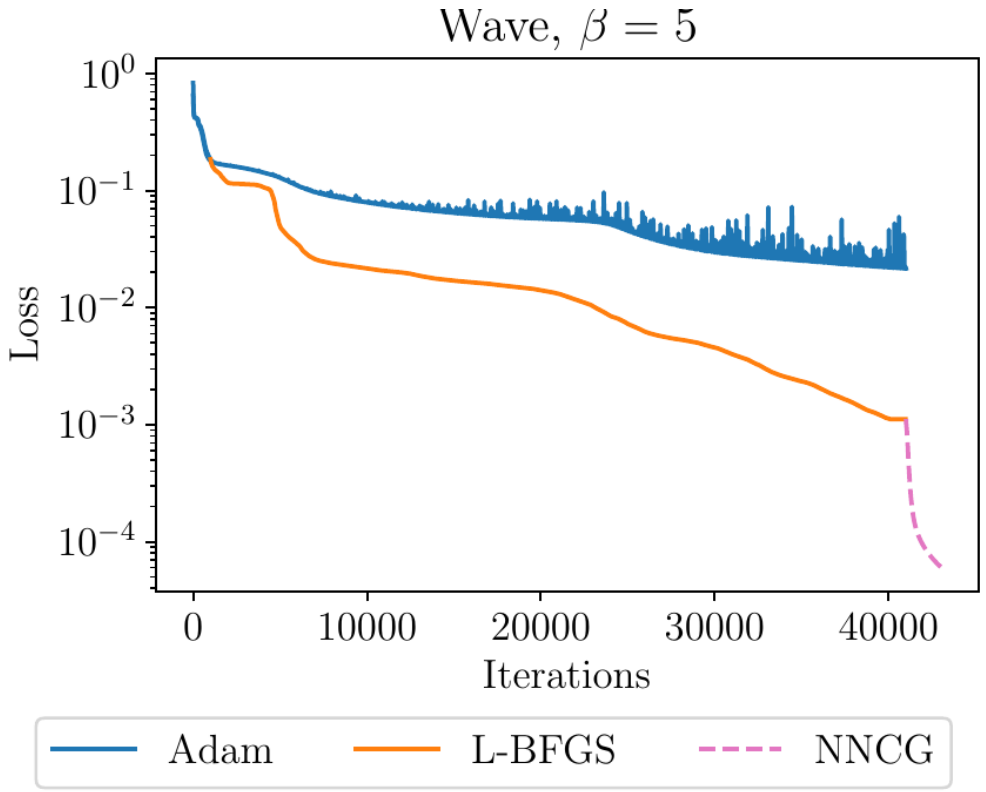
Causal training

$$\mathcal{L}_r(\theta) = \frac{1}{M}\sum_{i=1}^{M} w_i\,\mathcal{L}_r(t_i,\theta), \qquad w_i = \exp\Big(-\epsilon\sum_{k=1}^{i-1}\mathcal{L}_r(t_k,\theta)\Big)$$

- Standard training can reduce the residual at late times **before** the early times are right, then fit the PDE to a wrong state
- Split [0, *T*] into *M* time slices. Weight on slice *i* is small until the residual on **earlier** slices is small

Wang, Sankaran & Perdikaris, Respecting causality for training physics-informed neural networks, CMAME 2024

An ill-conditioned loss



- Gradient descent is slow in a long, narrow valley. Ratio of largest to smallest curvature, the **condition number**, determines the convergence rate
- Derivatives in the residual amplify high frequencies: $\partial^2/\partial x^2$ multiplies a mode $\sin(kx)$ by k^2. Curvatures spread over **many orders of magnitude**
- Wave equation: Adam stalls. Switching to **L-BFGS** lowers the loss by orders of magnitude. A Newton-type method (NNCG) lowers it further
- Standard practice: **Adam, then L-BFGS**

A smaller loss generally means a smaller error, so better optimizers pay off

Rathore, Lei, Frangella, Lu & Udell, Challenges in training PINNs: a loss landscape perspective, ICML 2024

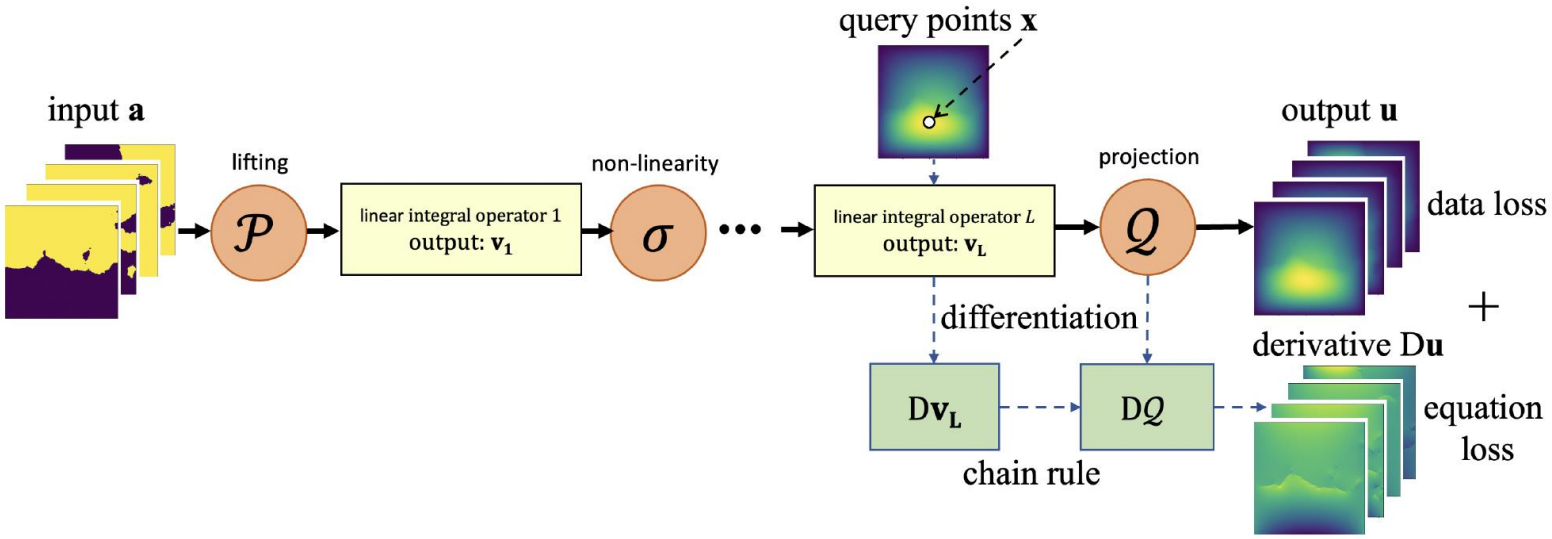
Training recipe for PINNs

- **Scale**: nondimensionalize the PDE and map inputs to $[-1, 1]$
- **Network**: tanh or sine activations; **Fourier features** for multiscale solutions
- **Conditions**: impose boundary conditions exactly when possible; otherwise balance the loss weights
- **Points**: resample collocation points; add points where the residual is large
- **Time**: causal weights, curriculum or time marching for long times and fast transport
- **Optimizer**: Adam, then L-BFGS
- **Check**: compare with a reference solver; a small residual does **not guarantee** a small error

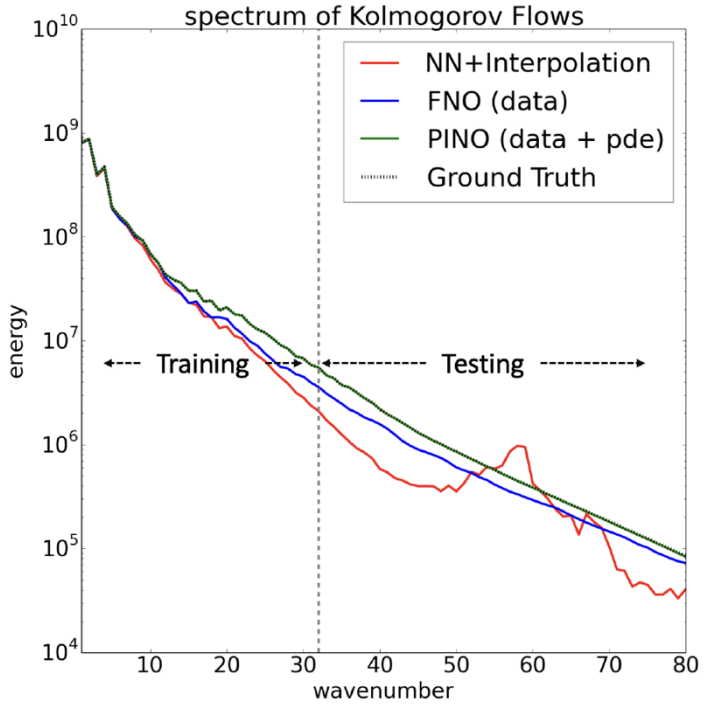
Each fix addresses one failure mode. Recent guides combine all of them.

Wang, Sankaran, Wang & Perdikaris, An expert's guide to training physics-informed neural networks, arXiv 2023

Physics-informed neural operators



$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N}\sum_{n=1}^{N} \left\| \mathcal{G}_\theta(a_n) - u_n \right\|^2}_{\text{data}} + \lambda \underbrace{\frac{1}{N}\sum_{n=1}^{N} \left\| \mathcal{R}\left(\mathcal{G}_\theta(a_n); a_n\right) \right\|^2}_{\text{PDE residual}}$$



- An operator $\mathcal{G}_\theta$ maps an input function *a* to the solution *u*. PINO adds the **residual of the prediction** to the data loss
- The residual can be evaluated on a **finer grid** than the data: on Kolmogorov flow, PINO follows the true spectrum beyond the training resolution (right)
- Physics-informed DeepONets train with the residual alone, without paired solutions

Li et al., Physics-informed neural operator, ACM/IMS J. Data Science 2024. Wang, Wang & Perdikaris, Science Advances 2021

Physics-informed losses in your project

- Good fits: **inverse problems** (unknown coefficients, sources or boundary values from sparse data), **filling in** fields from sparse sensors, a **residual loss** added to a U-Net or FNO surrogate
- On a grid, compute the residual with **finite differences** instead of autodiff
- Libraries: DeepXDE, NVIDIA PhysicsNeMo, jaxpi
- Baselines: a numerical solver at equal accuracy and the same network trained on data only
- Report the error against a reference solution, not only the residual loss
- BubbleML: the full boiling equations include phase change at a moving interface, so a residual loss is hard to write. Start with the heat equation in the solid or liquid

github.com/lululxvi/deepxde, github.com/NVIDIA/physicsnemo, github.com/PredictiveIntelligenceLab/jaxpi

Summary

- A PINN is a neural field trained so that the PDE residual, initial and boundary conditions vanish at sampled points
- Input derivatives come from autodiff; activations must be smooth (tanh, sine)
- Forward problem needs no solution data, but a PINN solves **one instance** and is usually slower than a solver
- With data and trainable coefficients, the same loss solves **inverse problems**: Burgers ν, Navier-Stokes coefficients, velocity and pressure from dye
- Training is the hard part: unbalanced loss terms, causality, ill-conditioning, fast transport
- Fixes: adaptive weights, causal weights, curriculum, Adam then L-BFGS
- Residual losses also train neural operators (PINO, physics-informed DeepONet)

After class

- Raissi, Perdikaris & Karniadakis, Physics-informed neural networks, JCP 2019: Sections 1-3 and Appendix A
- Karniadakis et al., Physics-informed machine learning, Nature Reviews Physics 2021
- Krishnapriyan et al., Characterizing possible failure modes in physics-informed neural networks, NeurIPS 2021
- Wang, Sankaran, Wang & Perdikaris, An expert's guide to training physics-informed neural networks, arXiv 2023
- Grossmann et al., Can physics-informed neural networks beat the finite element method?, IMA J. Appl. Math. 2024

Next, Lecture 15 (Thu, Nov 12): physics-constrained and hybrid learning

NEXT: HANDS-ON

From concepts to code

→ **Open the Lecture 14 notebook**

1. Derivatives of a network with autograd: tanh and ReLU
2. PINN for the 1D heat equation against the exact solution
3. PINN for Burgers' equation: prediction, error and loss terms
4. Convection at β = 1, 30 and 50: where Adam fails, where L-BFGS helps
5. Inverse problem: recover ν from 500 noisy measurements